

1. There are two nodes in a network that use stable storage and acknowledgment message passing for reliable communication. The stable storage of Node A contains the following record –

**Received message (In6); Transmitted message(Out3); Out:3 Ack:3 In:6.**

The stable storage of Node B contains the following record –

**Received message (In3); Transmitted message(Out6); Out:6 Ack:6 In:3.**

Now Node B sends a new message 7 to Node A. What will be in the stable storage of A and B if the message is received correctly, including a correctly received acknowledgement?

**Answer:**

**Node A: Received message (In7); Transmitted message(Out3); Out:3 Ack:3 In:7.**

**Node B:Received message (In3); Transmitted message(Out7); Out:7Ack:7In:3.**

2. Assume a large distributed system has many servers at different locations. The network connection between the servers is not reliable. There are millions of active users who regularly use this system. Why eventual consistency is more suitable than strong consistency for this system?

**Answer:**

Immediate strong consistency is impossible in case of a network partition. Moreover, as there are many servers and users, it is non-trivial to efficiently and reliably propagate and store data at any given time with unreliable network connection. Ensuring strong consistency will require relaxing availability and having lower latency according to the CAP theorem. As there are millions of users, availability is important to provide service to all these users with minimum latency. With eventual consistency, the updates will propagate to the other nodes and partitions of the network eventually, but at the cost of this relaxed consistency, the system will be able to offer higher availability in the event of network partition.

3. Consider a database storing the records of one million families, where each family is a row in a table. A unique ID, the location of the family's current residence, and the total household income per year of each family are stored in the database. The total household income of any family can be between 0 to 2 million, and it is an integer number (i.e., a whole number without any decimal value). For each of the following queries on this database, which index is the most suitable choice for efficiency? If there is any search key that needs to be specified for that index, mention the search key as well. If there's no index that can improve a query's performance, write 'no index' for that query. You need to explain the rationale of each of your answers.

Answer for each of the following queries -

- (i) find the IDs of the families where the total household income is less than 70,000;
- (ii) find the household income of the families who live within 5km of Melbourne central station.
- (iii) find the average household income of the families.

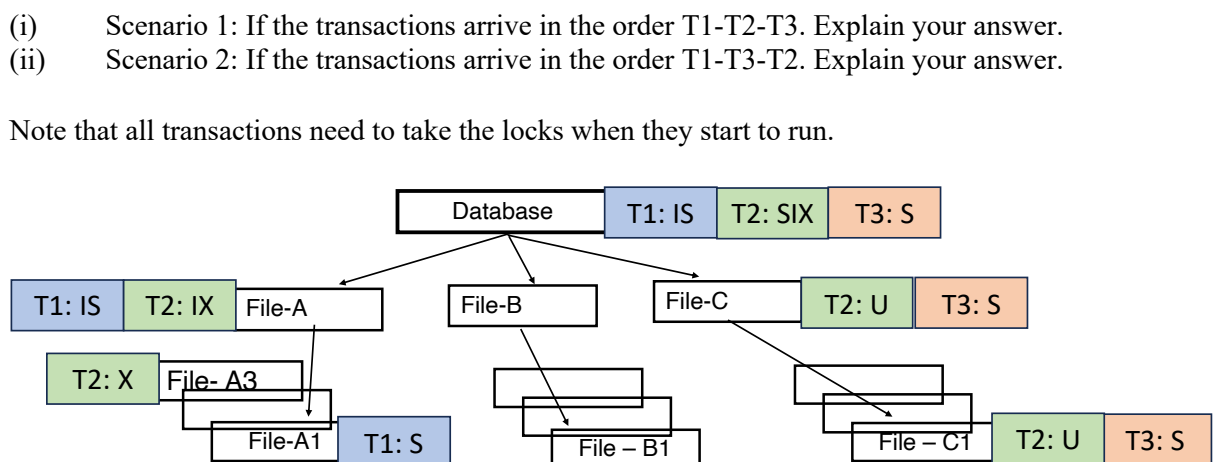
Answer:

- (i) B+ tree with the search key 'household income' as we want to find records with an income range
- (ii) R-tree or quad tree as the search is based on the distance of the location of the residence from another location.
- (iii) No index, as all the records are needed to be accessed.

4. When a database needs to join two tables using a page-oriented nested loop join algorithm, both tables are found to be in the main memory. Will it change how the cost of a cost-based optimiser calculates the cost of the query plan?

Answer: No, the cost calculation of a query plan will be same as if both tables were on disk.

5. Given the hierarchy of database objects and the corresponding granular locks in the following picture, which transactions can run concurrently from the beginning till end (that is, all operations and locks are compatible to run concurrently with another one) and which ones need to be delayed, answer for both scenarios –



Note that all transactions need to take the locks when they start to run.

A granular lock compatibility matrix is as follows –

| Compatibility Mode of Granular Locks |                                       |        |       |      |        |      |      |
|--------------------------------------|---------------------------------------|--------|-------|------|--------|------|------|
| Current                              | None                                  | IS     | IX    | S    | SIX    | U    | X    |
| Request                              | +/- (Next mode) + granted / - delayed |        |       |      |        |      |      |
| IS                                   | +(IS)                                 | +(IS)  | +(IX) | +(S) | +(SIX) | -(U) | -(X) |
| IX                                   | +(IX)                                 | +(IX)  | +(IX) | -(S) | -(SIX) | -(U) | -(X) |
| S                                    | +(S)                                  | +(S)   | -(IX) | +(S) | -(SIX) | -(U) | -(X) |
| SIX                                  | +(SIX)                                | +(SIX) | -(IX) | -(S) | -(SIX) | -(U) | -(X) |
| U                                    | +(U)                                  | +(U)   | -(IX) | +(U) | -(SIX) | -(U) | -(X) |
| X                                    | +(X)                                  | -(IS)  | -(IX) | -(S) | -(SIX) | -(U) | -(X) |

Answer:

- (i) Scenario 1:  
- T1, T2 can run parallel as IS and SIX are compatible at root, IS and IX are also compatible at A. T1 and T2 are taking S and X type locks at different nodes at the end, so no conflict there.

- T3 will be delayed. T3's S lock request is not compatible with T2's SIX lock at root.
- (ii) Scenario 2:
- T1, T3 can run in parallel as IS and S are compatible at root, then they are not accessing any common object.
  - T2 will be delayed as it's SIX lock is not compatible with T3's S lock at root.